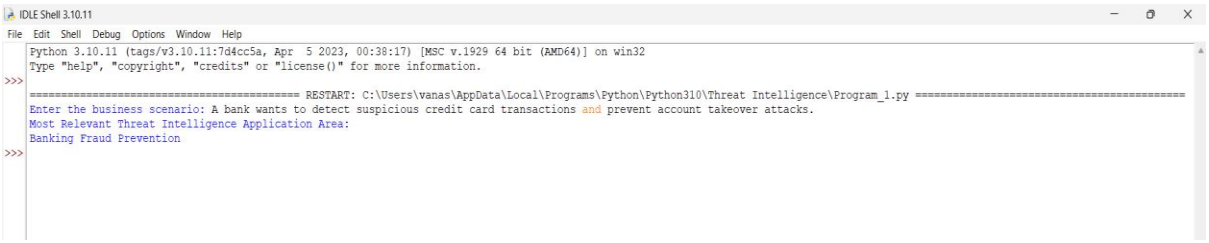
1. Write a Python program that reads a short business scenario and identifies the most relevant threat-intelligence application area for that industry context (e.g., banking fraud prevention, healthcare data protection, e-commerce fraud prevention).

Code:

```
def identify_application(scenario):
    scenario = scenario.lower()
    application_keywords = {
        "Banking Fraud Prevention": [
            "bank", "credit card", "transaction",
            "fraud", "payment", "account takeover"
        ],
        "Healthcare Data Protection": [
            "hospital", "healthcare", "patient",
            "medical", "health record", "ehr"
        ],
        "E-Commerce Fraud Prevention": [
            "e-commerce", "online shopping", "retailer",
            "customer order", "checkout", "online payment"
        ],
        "Cloud Security": [
            "cloud", "cloud storage", "virtual machine",
            "saas", "paas", "iaas"
        ],
        "Telecommunication Security": [
            "telecom", "mobile network", "subscriber",
            "5g", "4g", "cellular"
        ]
    }
    scores = {}
    for area, keywords in application_keywords.items():
        scores[area] = sum(
```

```
            1 for keyword in keywords if keyword in scenario
        )
    best_area = max(scores, key=scores.get)
    if scores[best_area] == 0:
        return "No specific application area identified."
    return best_area
scenario = input("Enter the business scenario: ")
print("Most Relevant Threat Intelligence Application Area:")
print(identify_application(scenario))
```

Output:



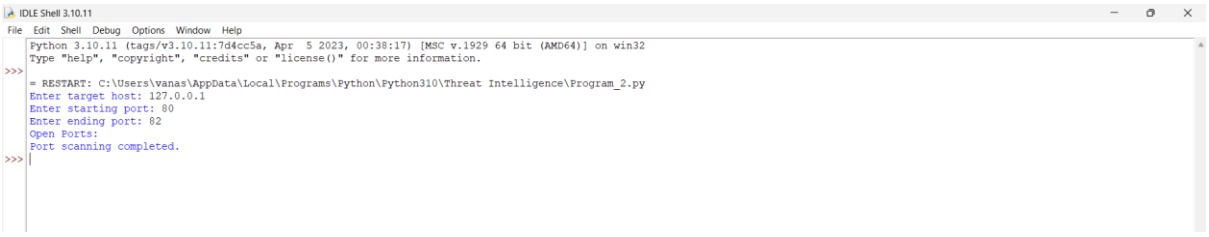
2. Write a Python program to perform a simple vulnerability assessment by implementing a port scanner that identifies open ports on a target host.

Code:

```
import socket
target = input("Enter target host: ")
start_port = int(input("Enter starting port: "))
end_port = int(input("Enter ending port: "))
print("Open Ports:")
for port in range(start_port, end_port + 1):
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.settimeout(0.5)
    try:
        result = sock.connect_ex((target, port))
        if result == 0:
            print(f"Port {port} is OPEN")
    except socket.gaierror:
```

```
        print("Invalid hostname.")
        break
    finally:
        sock.close()
print("Port scanning completed.")
```

Output:



3. Write a Python program to check a draft security policy document for required governance elements (scope, roles and responsibilities, enforcement, review cycle, approval authority) and report which are missing.

Code:

```
def check_security_policy(policy):
    required_elements = {
        "Scope": ["scope", "applies to"],
        "Roles and Responsibilities": [
            "roles and responsibilities", "responsibilities"
        ],
        "Enforcement": ["enforcement", "disciplinary action"],
        "Review Cycle": [
            "review cycle", "review annually", "review every year"
        ],
        "Approval Authority": [
            "approval authority", "approved by", "authorized by"
        ]
    }
    policy = policy.lower()
    missing = []
```

```
    for element, keywords in required_elements.items():
        if not any(word in policy for word in keywords):
            missing.append(element)
    if missing:
        print("Missing Governance Elements:")
        for element in missing:
            print("-", element)
    else:
        print("All required governance elements are present.")
print("Enter policy text (type END on a new line):")
lines = []
while True:
    line = input()
    if line.strip().upper() == "END":
        break
    lines.append(line)
policy = " ".join(lines)
check_security_policy(policy)
```

Output:



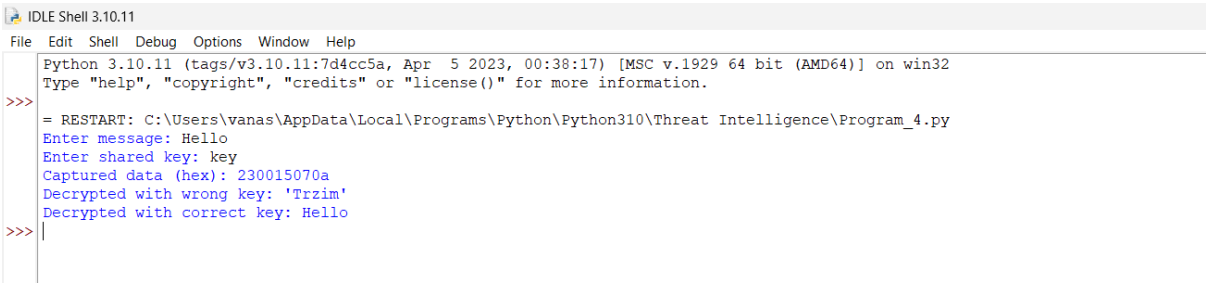
4. Write a Python program to demonstrate the confidentiality property a VPN tunnel provides, showing that data captured in transit is unreadable without the shared key, using a simple symmetric stream cipher as a stand-in for a real VPN's encryption.

Code:

```
def xor_cipher(data, key):
    result = bytearray()
```

```
    for i, byte in enumerate(data):
        result.append(byte ^ key[i % len(key)])
    return bytes(result)
message = input("Enter message: ")
key = input("Enter shared key: ")
if not key:
    print("Key cannot be empty.")
else:
    original = message.encode("utf-8")
    shared_key = key.encode("utf-8")
    encrypted = xor_cipher(original, shared_key)
    print("Captured data (hex):", encrypted.hex())
    wrong_key = b"wrong"
    wrong_decrypted = xor_cipher(encrypted, wrong_key)
    print("Decrypted with wrong key:",
        repr(wrong_decrypted.decode("utf-8", errors="replace")))
    decrypted = xor_cipher(encrypted, shared_key)
    print("Decrypted with correct key:",
        decrypted.decode("utf-8"))
```

Output:

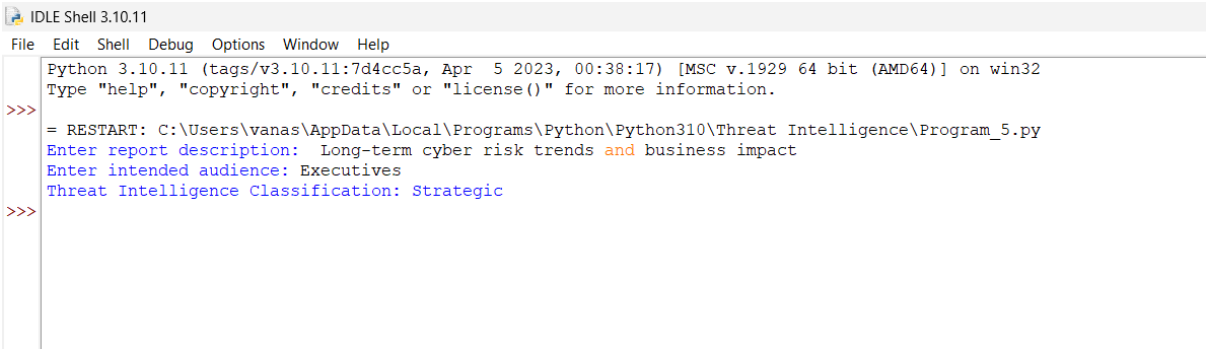


5. Write a Python program using a keyword-based classifier that reads a description of a threat- intelligence report and its intended audience, and classifies it as Strategic, Tactical, or Operational.

Code:

```
def classify_report(description, audience):
    text = (description + " " + audience).lower()
    keywords = {
        "Strategic": [
            "business leaders", "executives", "risk trends",
            "long-term", "business impact", "management"
        ],
        "Tactical": [
            "security team", "attack techniques", "mitre",
            "ttp", "adversary methods", "defensive strategy"
        ],
        "Operational": [
            "soc analyst", "incident response", "ip address",
            "malicious domain", "hash", "active attack",
            "incident details", "ioc"
        ]
    }
    scores = {}
    for category, words in keywords.items():
        scores[category] = sum(
            1 for word in words if word in text
        )
    if max(scores.values()) == 0:
        return "Unclassified"
    return max(scores, key=scores.get)
description = input("Enter report description: ")
audience = input("Enter intended audience: ")
print("Threat Intelligence Classification:",
      classify_report(description, audience))
```

Output:



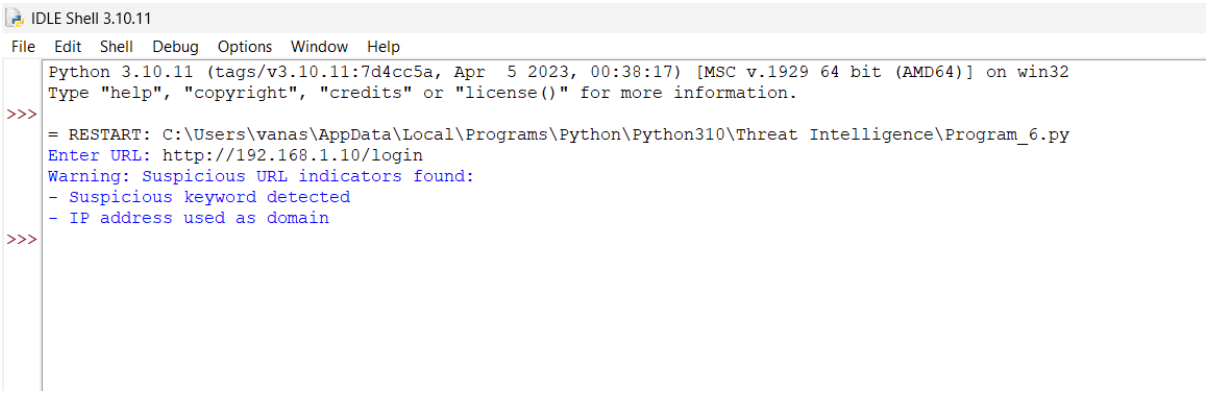
6. Write a Python program to detect phishing URLs by checking a given URL against common phishing indicators (e.g., suspicious keywords, IP-based domains, excessive subdomains).

Code:

```
from urllib.parse import urlparse
import ipaddress
def detect_phishing(url):
    suspicious_words = [
        "login", "verify", "secure", "update",
        "account", "password", "banking"
    ]
    indicators = []
    parsed = urlparse(
        url if "://" in url else "//" + url
    )
    host = (parsed.hostname or "").lower()
    if not host:
        return ["Invalid URL or missing hostname"]
    if any(word in url.lower() for word in suspicious_words):
        indicators.append("Suspicious keyword detected")
    try:
        ipaddress.ip_address(host)
        indicators.append("IP address used as domain")
    except ValueError:
```

```
        pass
    if len(host.split(".")) > 4:
        indicators.append("Excessive subdomains")
    if "@" in parsed.netloc:
        indicators.append("Contains @ symbol in authority")
    return indicators
url = input("Enter URL: ")
results = detect_phishing(url)
if results:
    print("Warning: Suspicious URL indicators found:")
    for item in results:
        print("-", item)
else:
    print("No obvious phishing indicators found.")
```

Output:



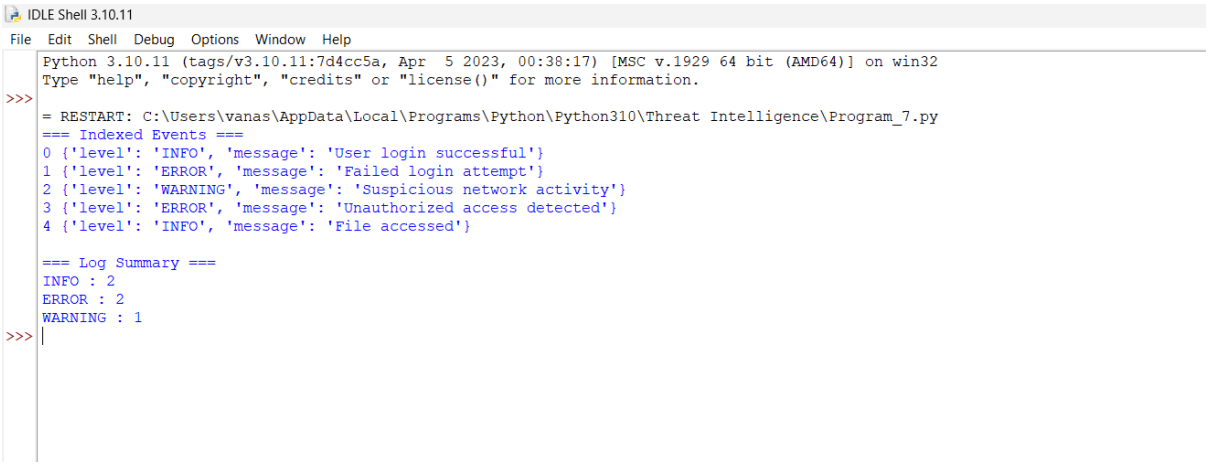
7. Write a Python program to reproduce the three ELK Stack roles: Logstash-style parsing of raw log lines into structured events, Elasticsearch-style indexing, and Kibana-style aggregation.

Code:

```
raw_logs = [
    "INFO User login successful",
    "ERROR Failed login attempt",
    "WARNING Suspicious network activity",
```

```
    "ERROR Unauthorized access detected",
    "INFO File accessed"
]
events = []
for log in raw_logs:
    parts = log.split(" ", 1)
    event = {
        "level": parts[0],
        "message": parts[1]
    }
    events.append(event)
index = {}
for i, event in enumerate(events):
    index[i] = event
summary = {}
for event in index.values():
    level = event["level"]
    summary[level] = summary.get(level, 0) + 1
print("=== Indexed Events ===")
for event_id, event in index.items():
    print(event_id, event)
print("\n=== Log Summary ===")
for level, count in summary.items():
    print(level, ":", count)
```

Output:

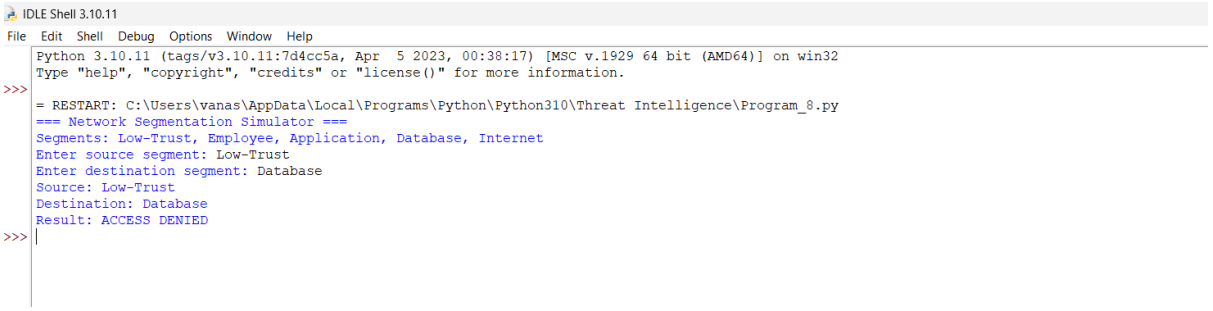


8. Write a Python program to model a segmented network's access policy and demonstrate how network segmentation blocks lateral movement from a compromised low-trust segment into a sensitive one.

Code:

```
allowed_connections = {
    ("Low-Trust", "Internet"),
    ("Employee", "Application"),
    ("Application", "Database")
}
def check_access(source, destination):
    if (source, destination) in allowed_connections:
        return "ACCESS ALLOWED"
    else:
        return "ACCESS DENIED"
print("=== Network Segmentation Simulator ===")
print("Segments: Low-Trust, Employee, Application, Database, Internet")
source = input("Enter source segment: ")
destination = input("Enter destination segment: ")
result = check_access(source, destination)
print("Source:", source)
print("Destination:", destination)
print("Result:", result)
```

Output:


IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\vanas\AppData\Local\Programs\Python\Python310\Threat Intelligence\Program_8.py
=== Network Segmentation Simulator ===
Segments: Low-Trust, Employee, Application, Database, Internet
Enter source segment: Low-Trust
Enter destination segment: Database
Source: Low-Trust
Destination: Database
Result: ACCESS DENIED
>>>